Casos de prueba de la API

Sistema de cadetería — Testing de endpoints

Fecha de ejecución: 6 de octubre de 2026

1. Objetivo

Este documento describe las pruebas realizadas sobre la API del sistema de cadetería. El objetivo es comprobar que cada endpoint responde bien cuando recibe datos correctos y que, cuando recibe datos incorrectos o un pedido sin autorización, devuelve el código de error HTTP que corresponde junto con un mensaje claro.

2. Alcance

Se probaron 5 endpoints con 3 casos cada uno: un caso correcto y dos casos de error. En total son 15 casos de prueba.

N.º	Endpoint	Qué hace
1	`POST /api/auth/login/cadete`	Inicio de sesión de la aplicación de cadetes. Recibe usuario (el DNI) y contraseña, y devuelve el token que se usa en el resto de los pedidos.
2	`GET /api/cadetes/me`	Devuelve los datos del cadete que inició sesión. Se usa para probar el control del token.
3	`GET /api/admin/cadetes/{id}`	Consulta de un cadete desde el panel de administración. Solo para administradores.
4	`PATCH /api/cadetes/me/estado`	El cadete cambia su estado (LIBRE, OCUPADO o DESCONECTADO).
5	`GET /api/pedidos/me/{id}`	Detalle de un pedido del cadete que inició sesión. Solo para cadetes.

Los casos de error cubren los cuatro códigos más comunes de una API:

- 400 Bad Request: los datos enviados no son válidos.
- 401 Unauthorized: no hay sesión, el token es inválido o las credenciales son incorrectas.
- 403 Forbidden: hay sesión, pero el usuario no tiene permiso para esa ruta.
- 404 Not Found: el recurso pedido no existe.

3. Entorno y herramientas

- API: backend del sistema de cadetería (Java con Spring Boot y base de datos MySQL).
- Herramienta: Postman, con la colección «instituto-15-casos.postman_collection.json» que acompaña este documento.
- Datos: carga de demostración del sistema (un administrador, cadetes y pedidos de ejemplo).
- Usuarios de prueba: administrador «admin» y cadete «30111222» (el usuario del cadete es su DNI).
- Ejecución registrada en este documento: 6 de octubre de 2026, sobre una instalación local del backend con una base de datos de prueba.

4. Formato de las respuestas de error

Todos los errores de la API tienen la misma forma. Además del código HTTP, el cuerpo de la respuesta trae un código propio del sistema y un mensaje en castellano:

```
{
  "timestamp": "2026-10-06T17:10:10.478Z",
  "status": 401,
  "error": "Unauthorized",
  "codigo": "CREDENCIALES_INVALIDAS",
  "message": "Usuario o contraseña incorrectos.",
  "fieldErrors": null
}
```

Campo	Significado
`status`	Código HTTP de la respuesta (400, 401, 403, 404...).
`error`	Nombre estándar del código HTTP.
`codigo`	Código propio del sistema, que identifica el tipo de error.
`message`	Explicación del error en castellano, para mostrar al usuario.
`fieldErrors`	En los errores de validación, el detalle campo por campo. En el resto, null.

5. Cómo ejecutar las pruebas

1. Abrir Postman, elegir Import y seleccionar el archivo de la colección.
2. En la pestaña Variables de la colección, cargar en «baseUrl» la dirección de la API y, si hace falta, los usuarios de prueba.
3. Elegir Run collection y ejecutar la colección completa, en el orden en que está.
4. Revisar el resultado: cada caso verifica solo el código HTTP y el código de error de la respuesta.

La colección incluye tres pedidos de preparación (P1, P2 y P3) que no cuentan como casos: inician sesión y guardan los tokens y los identificadores que después usan los casos. Por eso la colección se tiene que ejecutar completa y en orden.

6. Resumen de los casos

N.º	Endpoint	Caso	HTTP	Código de error
01	`POST /api/auth/login/cadete`	Login correcto	**200**	—
02	`POST /api/auth/login/cadete`	Contraseña incorrecta	**401**	`CREDENCIALES_INVALIDAS`
03	`POST /api/auth/login/cadete`	Letras en un campo numérico	**400**	`VALIDACION`
04	`GET /api/cadetes/me`	Token válido	**200**	—
05	`GET /api/cadetes/me`	Token mal formado	**401**	`NO_AUTENTICADO`
06	`GET /api/cadetes/me`	Token vencido	**401**	`NO_AUTENTICADO`
07	`GET /api/admin/cadetes/{id}`	Cadete que existe	**200**	—
08	`GET /api/admin/cadetes/{id}`	Cadete que no existe	**404**	`NO_ENCONTRADO`

N.º	Endpoint	Caso	HTTP	Código de error
09	`GET /api/admin/cadetes/{id}`	Token de cadete en una ruta de administrador	**403**	`SIN_PERMISO`
10	`PATCH /api/cadetes/me/estado`	Estado válido	**200**	—
11	`PATCH /api/cadetes/me/estado`	Estado que no existe	**400**	`VALIDACION`
12	`PATCH /api/cadetes/me/estado`	Sin token	**401**	`NO_AUTENTICADO`
13	`GET /api/pedidos/me/{id}`	Pedido propio	**200**	—
14	`GET /api/pedidos/me/{id}`	Pedido que no existe	**404**	`NO_ENCONTRADO`
15	`GET /api/pedidos/me/{id}`	Token de administrador en una ruta de cadete	**403**	`SIN_PERMISO`

7. Detalle de los casos

7.1. POST /api/auth/login/cadete

Inicio de sesión de la aplicación de cadetes. Recibe usuario (el DNI) y contraseña, y devuelve el token que se usa en el resto de los pedidos.

Caso 01 — Login correcto

Objetivo	Verificar que un cadete con usuario (DNI) y contraseña correctos obtiene su token de acceso.
Endpoint	`POST /api/auth/login/cadete`
Precondición	El cadete de prueba existe y está activo.
Datos de entrada	`{ "username": "30111222", "password": "cadete123" }`
Resultado esperado	HTTP 200. Devuelve el token, tokenType "Bearer" y tipo "CADETE".

Resultado obtenido	`HTTP 200`
	`{ "token": "eyJhbGciOi...", "tokenType": "Bearer",`
	`"tipo": "CADETE", "expiresAt": "2026-10-07T05:10:22Z" }`

Estado	**Aprobado**

Caso 02 — Contraseña incorrecta

Objetivo	Verificar que no se entrega un token cuando la contraseña no coincide.
Endpoint	`POST /api/auth/login/cadete`
Precondición	El cadete de prueba existe.
Datos de entrada	`{ "username": "30111222", "password": "clave-incorrecta" }`
Resultado esperado	HTTP 401 · código CREDENCIALES_INVALIDAS. Rechaza el ingreso sin indicar cuál de los dos datos está mal.

Resultado obtenido	`HTTP 401`
	`{ "status": 401, "error": "Unauthorized",`
	`"codigo": "CREDENCIALES_INVALIDAS",`
	`"message": "Usuario o contraseña incorrectos." }`

Estado	**Aprobado**

Caso 03 — Letras en un campo numérico

Objetivo	Verificar que la API valida el tipo de dato: el campo versionApp es numérico y se le envían letras.
Endpoint	`POST /api/auth/login/cadete`
Precondición	Ninguna.
Datos de entrada	`{ "username": "30111222", "password": "cadete123", "versionApp": "abc" }`
Resultado esperado	HTTP 400 · código VALIDACION. Rechaza el pedido e indica qué campo tiene el formato incorrecto.
Resultado obtenido	`HTTP 400` `{ "status": 400, "error": "Bad Request", "codigo": "VALIDACION", "message": "El dato «versionApp» no tiene el formato correcto.", "fieldErrors": { "versionApp": "Formato incorrecto" } }`
Estado	**Aprobado**

7.2. GET /api/cadetes/me

Devuelve los datos del cadete que inició sesión. Se usa para probar el control del token.

Caso 04 — Token válido

Objetivo	Verificar que con un token vigente el cadete obtiene sus propios datos.
Endpoint	`GET /api/cadetes/me`
Precondición	Token obtenido en el caso 01.
Datos de entrada	`Authorization: Bearer <token del caso 01>`
Resultado esperado	HTTP 200. Devuelve el perfil del cadete que inició sesión.
Resultado obtenido	`HTTP 200` `{ "id": "034bd6fc-...", "nombre": "Juan", "apellido": "Pérez", "dni": "30111222", "telefono": "3813010001", ... }`
Estado	**Aprobado**

Caso 05 — Token mal formado

Objetivo	Verificar que un token que no tiene el formato de un JWT es rechazado.
Endpoint	`GET /api/cadetes/me`
Precondición	Ninguna.
Datos de entrada	`Authorization: Bearer esto.no.esuntoken`
Resultado esperado	HTTP 401 · código NO_AUTENTICADO. Rechaza el pedido por no estar autenticado.
Resultado obtenido	`HTTP 401` `{ "status": 401, "error": "Unauthorized", "codigo": "NO_AUTENTICADO", "message": "Tu sesión venció o no iniciaste sesión." }`
Estado	**Aprobado**

Caso 06 — Token vencido

Objetivo	Verificar que un token que ya no está vigente es rechazado.
Endpoint	`GET /api/cadetes/me`

Precondición	Token de una sesión anterior del mismo cadete (paso de preparación P2). La API admite una sola sesión por cadete: al iniciar sesión otra vez (caso 01), el token anterior deja de valer.
Datos de entrada	`Authorization: Bearer <token de la sesión anterior>`
Resultado esperado	HTTP 401 · código NO_AUTENTICADO. Rechaza el pedido igual que con un token vencido por tiempo.

Resultado obtenido	`HTTP 401`
	`{ "status": 401, "error": "Unauthorized",`
	`  "codigo": "NO_AUTENTICADO",`
	`  "message": "Tu sesión venció o no iniciaste sesión." }`

Estado	**Aprobado**

7.3. GET /api/admin/cadetes/{id}

Consulta de un cadete desde el panel de administración. Solo para administradores.

Caso 07 — Cadete que existe

Objetivo	Verificar que un administrador puede consultar los datos de un cadete por su id.
Endpoint	`GET /api/admin/cadetes/{id}`
Precondición	Token de administrador (paso de preparación P1) e id del cadete (se toma del caso 04).
Datos de entrada	`GET /api/admin/cadetes/034bd6fc-48c3-485c-86af-97ec920da2be` `Authorization: Bearer <token de admin>`
Resultado esperado	HTTP 200. Devuelve los datos del cadete pedido.

Resultado obtenido	`HTTP 200`
	`{ "id": "034bd6fc-...", "nombre": "Juan", "apellido": "Pérez",`
	`  "dni": "30111222", ... }`

Estado	**Aprobado**

Caso 08 — Cadete que no existe

Objetivo	Verificar la respuesta cuando se pide un cadete con un id inexistente.
Endpoint	`GET /api/admin/cadetes/{id}`
Precondición	Token de administrador.
Datos de entrada	`GET /api/admin/cadetes/no-existe-99999` `Authorization: Bearer <token de admin>`
Resultado esperado	HTTP 404 · código NO_ENCONTRADO. Informa que el cadete no existe.

Resultado obtenido	`HTTP 404`
	`{ "status": 404, "error": "Not Found",`
	`  "codigo": "NO_ENCONTRADO",`
	`  "message": "Cadete no encontrado: no-existe-99999" }`

Estado	**Aprobado**

Caso 09 — Token de cadete en una ruta de administrador

Objetivo	Verificar que un usuario autenticado pero sin el rol necesario no accede a las rutas del panel.
Endpoint	`GET /api/admin/cadetes/{id}`
Precondición	Token de cadete (caso 01).

Datos de entrada	`GET /api/admin/cadetes/034bd6fc-48c3-485c-86af-97ec920da2be` `Authorization: Bearer <token de cadete>`
Resultado esperado	HTTP 403 · código SIN_PERMISO. Rechaza el pedido por falta de permiso.

Resultado obtenido	`HTTP 403`
	`{ "status": 403, "error": "Forbidden",`
	`  "codigo": "SIN_PERMISO",`
	`  "message": "No tenés permiso para hacer eso." }`

Estado	**Aprobado**

7.4. PATCH /api/cadetes/me/estado

El cadete cambia su estado (LIBRE, OCUPADO o DESCONECTADO).

Caso 10 — Estado válido

Objetivo	Verificar que el cadete puede cambiar su estado a uno de los valores permitidos.
Endpoint	`PATCH /api/cadetes/me/estado`
Precondición	Token de cadete (caso 01).
Datos de entrada	`Authorization: Bearer <token de cadete>` `{ "estadoId": "DESCONECTADO" }`
Resultado esperado	HTTP 200. Guarda el estado y devuelve los datos del cadete.
Resultado obtenido	`HTTP 200` `{ "id": "034bd6fc-...", "nombre": "Juan", "apellido": "Pérez", ... }`
Estado	**Aprobado**

Caso 11 — Estado que no existe

Objetivo	Verificar que se rechaza un estado fuera de los valores permitidos (LIBRE, OCUPADO, DESCONECTADO).
Endpoint	`PATCH /api/cadetes/me/estado`
Precondición	Token de cadete (caso 01).
Datos de entrada	`Authorization: Bearer <token de cadete>` `{ "estadoId": "VOLANDO" }`
Resultado esperado	HTTP 400 · código VALIDACION. Rechaza el pedido e indica los valores aceptados.

Resultado obtenido	`HTTP 400`
	`{ "status": 400, "error": "Bad Request", "codigo": "VALIDACION",`
	`  "message": "Revisá los datos: El estado tiene que ser LIBRE,`
	`  OCUPADO o DESCONECTADO.",`
	`  "fieldErrors": { "estadoId": "El estado tiene que ser ..." } }`

Estado	**Aprobado**

Caso 12 — Sin token

Objetivo	Verificar que no se puede modificar el estado sin haber iniciado sesión.
Endpoint	`PATCH /api/cadetes/me/estado`
Precondición	Ninguna.
Datos de entrada	`(sin encabezado Authorization)` `{ "estadoId": "LIBRE" }`
Resultado esperado	HTTP 401 · código NO_AUTENTICADO. Rechaza el pedido por no estar autenticado.

Resultado obtenido	`HTTP 401`
	`{ "status": 401, "error": "Unauthorized",`
	`  "codigo": "NO_AUTENTICADO",`
	`  "message": "Tu sesión venció o no iniciaste sesión." }`

Estado	**Aprobado**

7.5. GET /api/pedidos/me/{id}

Detalle de un pedido del cadete que inició sesión. Solo para cadetes.

Caso 13 — Pedido propio

Objetivo	Verificar que el cadete puede ver el detalle de un pedido que le pertenece.
Endpoint	`GET /api/pedidos/me/{id}`
Precondición	Token de cadete (caso 01) e id de un pedido propio (paso de preparación P3, que lo toma del historial).
Datos de entrada	`GET /api/pedidos/me/demo-pedido-01` `Authorization: Bearer <token de cadete>`
Resultado esperado	HTTP 200. Devuelve el detalle del pedido.

Resultado obtenido	`HTTP 200`
	`{ "id": "demo-pedido-01", "numero": 9100001,`
	`  "clienteNombre": "Marcos Herrera",`
	`  "origenDireccion": "25 de Mayo 231, Monteros", ... }`

Estado	**Aprobado**

Caso 14 — Pedido que no existe

Objetivo	Verificar la respuesta cuando se pide un pedido con un id inexistente.
Endpoint	`GET /api/pedidos/me/{id}`
Precondición	Token de cadete (caso 01).
Datos de entrada	`GET /api/pedidos/me/no-existe-99999` `Authorization: Bearer <token de cadete>`
Resultado esperado	HTTP 404 · código NO_ENCONTRADO. Informa que el pedido no existe.

Resultado obtenido	`HTTP 404`
	`{ "status": 404, "error": "Not Found",`
	`  "codigo": "NO_ENCONTRADO",`
	`  "message": "Pedido no encontrado: no-existe-99999" }`

Estado	**Aprobado**

Caso 15 — Token de administrador en una ruta de cadete

Objetivo	Verificar que las rutas de la aplicación de cadetes no aceptan un token con otro rol.
Endpoint	`GET /api/pedidos/me/{id}`
Precondición	Token de administrador (paso de preparación P1).
Datos de entrada	`GET /api/pedidos/me/demo-pedido-01` `Authorization: Bearer <token de admin>`
Resultado esperado	HTTP 403 · código SIN_PERMISO. Rechaza el pedido por falta de permiso.

Resultado obtenido	`HTTP 403`
	`{ "status": 403, "error": "Forbidden",`
	`  "codigo": "SIN_PERMISO",`
	`  "message": "No tenés permiso para hacer eso." }`

Estado	**Aprobado**

8. Resultado

Los 15 casos dieron el resultado esperado: los 5 casos correctos respondieron con código 200 y los 10 casos de error respondieron con el código HTTP y el código de error previstos.

Concepto	Cantidad
Casos ejecutados	15
Casos aprobados	15
Casos fallidos	0
Casos correctos (200)	5
Casos de error 400	2
Casos de error 401	4
Casos de error 403	2
Casos de error 404	2

9. Observaciones

- Token vencido (caso 06): se usa el token de una sesión anterior del mismo cadete, que deja de valer cuando el cadete vuelve a iniciar sesión. La API responde igual que ante un token vencido por tiempo: 401 NO_AUTENTICADO.
- Letras en el usuario: el campo usuario del inicio de sesión es de texto, por lo que escribir letras no es un error de formato; la API responde 401 por credenciales incorrectas. El control de que el DNI sea numérico lo hace la aplicación antes de enviar. Para probar el error de formato (400) se usó el campo numérico versionApp (caso 03).
- Los identificadores de cadetes y pedidos son de texto, no numéricos: un identificador con cualquier valor que no exista devuelve 404.
- Los valores de los identificadores que figuran en el detalle (por ejemplo «034bd6fc-...» o «demo-pedido-01») corresponden a los datos de demostración de la ejecución registrada y pueden cambiar en otra instalación; la colección los obtiene sola en cada ejecución.